Backends

Simon Frost

Table of contents

1 Overview 1

2 One model, every backend 2

3 Apple GPUs with Metal 3

4 Which backend supports what 4

5 Sparse storage for invasions 4

6 Choosing a backend 5

1 Overview

A model in `CellularAutomataCore.jl` says nothing about where its state lives or how its cells are visited; that is the backend, chosen when the problem is solved:

backend	state	best for
`ReferenceBackend()`	plain arrays, boundaries applied on every access	checking the others
`ArrayBackend(; threaded)`	padded arrays, optionally threaded	most models
`SparseBackend(background; tilesize)`	only tiles that differ from the background	invasions across large, mostly empty landscapes
`GPUBackend(device)`	arrays on a GPU (or any KernelAbstractions device)	very large lattices

Every backend gives exactly the same result for the same problem, stochastic rules included, because random numbers are a function of the seed, the cell, the time and the draw number rather than of the order of computation. The test suite checks every backend against `ReferenceBackend` on a battery of problems.

```
ENV["GKSwstype"] = "100"
using CellularAutomataCore
using Plots
using Random
using KernelAbstractions
```

2 One model, every backend

A stochastic SIR epidemic, solved on each backend:

```
sir = LocalRule(VonNeumann(1)) do cell, nbrs, ctx
    if cell == 0
        return bernoulli(ctx, 1 - (1 - ctx.p.β)^count(==(1), nbrs), 1) ? Int8(1) :
        Int8(0)
    elseif cell == 1
        return bernoulli(ctx, ctx.p.γ, 2) ? Int8(2) : Int8(1)
    end
    return cell
end
n = 256
u0 = zeros(Int8, n, n)
u0[n ÷ 2, n ÷ 2] = 1
prob = CAProblem(sir, u0, 300, (β=0.6, γ=0.2); seed=1)

backends = ["Reference" ⇒ ReferenceBackend(),
            "Array, serial" ⇒ ArrayBackend(threaded=false),
            "Array, threaded" ⇒ ArrayBackend(threaded=true),
            "Sparse" ⇒ SparseBackend(Int8(0); tilesize=32),
            "GPU (KernelAbstractions CPU)" ⇒ GPUBackend(KernelAbstractions.CPU())]
results = map(backends) do (name, backend)
    solve(prob; backend, save_everystep=false)              # compile
    t = @elapsed sol = solve(prob; backend, save_everystep=false)
    (name=name, seconds=round(t; digits=3), final=final_state(sol))
end
[(r.name, r.seconds) for r in results]
```

```
5-element Vector{Tuple{String, Float64}}:
 ("Reference", 0.351)
 ("Array, serial", 0.157)
 ("Array, threaded", 0.07)
 ("Sparse", 0.145)
 ("GPU (KernelAbstractions CPU)", 0.112)
```

```
all(r.final == results[1].final for r in results), Threads.nthreads()
```

```
(true, 8)
```

All five agree exactly. The second number is the thread count this document was rendered with.

The KernelAbstractions CPU device runs the GPU code path on the CPU, which is how it is tested without a GPU. On a machine with a GPU, load its package and pass its device, or let `GPUBackend` take the device from an array:

```
using CUDA                       # or AMDGPU, Metal, oneAPI
backend = GPUBackend(CUDABackend())
backend = GPUBackend(CuArray(zeros(1)))   # the same, from an array
```

GPU rules must be usable inside a kernel: states and parameters that are plain bits types (numbers, `Bool`s, isbits structs, `NamedTuple`s of them), and functions that do not allocate. `Symbol` states and graph spaces run on the CPU backends only.

3 Apple GPUs with Metal

On a Mac, `using Metal` makes the Apple GPU available as `MetalBackend()`. Metal has no double precision, so a GPU-ready rule uses `Float32` parameters and literals: `bernoulli` then draws in single precision, and `Stochastic(0.5f0)`, `KernelField(...; precision=Float32)` and `Lenia(; precision=Float32)` keep everything in `Float32`. The code below runs only where Metal is available.

```
has_metal = Sys.isapple() && Base.find_package("Metal") !== nothing
if has_metal
    @eval using Metal
    has_metal = Metal.functional()
end
has_metal
```

```
true
```

```
if has_metal
    metal = GPUBackend(MetalBackend())
    sir32 = LocalRule(VonNeumann(1)) do c, n, ctx
        c == Int8(0) && return bernoulli(ctx, 1.0f0 - (1.0f0 -
ctx.p.β)^count(==(Int8(1)), n), 1) ?
                              Int8(1) : Int8(0)
        c == Int8(1) && return bernoulli(ctx, ctx.p.γ, 2) ? Int8(2) : Int8(1)
        return c
    end
    p32 = CAProblem(sir32, u0, 300, (β=0.6f0, γ=0.2f0); seed=1)
    same = final_state(solve(p32; backend=metal, save_everystep=false)) ==
           final_state(solve(p32; backend=ArrayBackend(), save_everystep=false))
    println("Float32 SIR on Metal identical to the CPU: ", same)
    lenia = CAProblem(Lenia(; precision=Float32), rand(Xoshiro(3), Float32, 512,
512), 30)
```

```
    for (name, backend) in ["CPU, threaded" ⇒ ArrayBackend(threaded=true), "Metal"
    ⇒ metal]
        integ = init(lenia; backend, save_everystep=false)
        step!(integ, 2)
        t = @elapsed step!(integ, 10)
        println(rpad(name, 14), round(1000t / 10; digits=1), " ms per Lenia step
        (512², R = 13)")
    end
else
    println("Metal is not available here; on an Apple-silicon Mac, add Metal to the
    environment.")
end
```

```
Float32 SIR on Metal identical to the CPU: true
CPU, threaded 22.2 ms per Lenia step (512², R = 13)
Metal         2.5 ms per Lenia step (512², R = 13)
```

The epidemic is identical on the GPU, because its random numbers do not depend on the order of computation. Lenia's 27×27 kernel makes each step compute-heavy, which is where a GPU pays off; Float32 Lenia on the GPU agrees with the CPU to rounding, since GPU implementations of `exp` differ in the last bits.

4 Which backend supports what

```
schemes = [Synchronous(), Stochastic(0.5), Checkerboard(), RandomSequential(),
Margolus()]
names_ = ["Reference", "Array", "Sparse", "GPU"]
bs = [ReferenceBackend(), ArrayBackend(), SparseBackend(0),
GPUBackend(KernelAbstractions.CPU())]
[(name, [string(nameof(typeof(s))) for s in schemes if supports(b, s)]) for (name,
b) in zip(names_, bs)]
```

```
4-element Vector{Tuple{String, Vector{String}}}:
 ("Reference", ["Synchronous", "Stochastic", "Checkerboard", "RandomSequential",
 "Margolus"])
 ("Array", ["Synchronous", "Stochastic", "Checkerboard", "RandomSequential",
 "Margolus"])
 ("Sparse", ["Synchronous", "Stochastic"])
 ("GPU", ["Synchronous", "Stochastic", "Checkerboard"])
```

5 Sparse storage for invasions

`SparseBackend` cuts the lattice into tiles and stores and updates only tiles that hold a cell differing from the background state. Early in an invasion that is a tiny fraction of the landscape, so the cost per step follows the infested area rather than the lattice size. The rule must leave a background

cell with all-background neighbours unchanged; the backend checks a sample of cells when the problem is set up.

```
n = 2048
v = zeros(Int8, n, n)
v[n ÷ 2, n ÷ 2] = 1
large = CAProblem(sir, v, 120, (β=0.6, γ=0.2); seed=2)
for (name, backend) in ["Array, threaded" ⇒ ArrayBackend(threaded=true),
                        "Sparse" ⇒ SparseBackend(Int8(0); tilesize=32)]
    integ = init(large; backend, save_everystep=false)
    step!(integ)  # compile
    t = @elapsed step!(integ, 119)
    extra = backend isa SparseBackend ? ", $(stored_tiles(integ)) of $(cld(n,
32)^2) tiles stored" : ""
    println(rpad(name, 18), round(1000t / 119; digits=2), " ms per step", extra)
end
```

```
Array, threaded   11.04 ms per step
Sparse            0.42 ms per step, 32 of 4096 tiles stored
```

After 120 steps the epidemic covers a small part of the 2048 × 2048 lattice: the sparse backend stores a few dozen of the 4096 tiles and is many times faster per step than updating every cell (the timings are from the machine that rendered this document). As the epidemic spreads, the advantage shrinks; once most tiles are active, the dense backend is faster.

6 Choosing a backend

- Start with the default `ArrayBackend()`: threaded when Julia has more than one thread, any scheme, any space.
- For invasions or outbreaks on large landscapes that are mostly in one state, try `SparseBackend(background)`.
- For very large lattices with isbits states and synchronous, stochastic or checkerboard updating, use `GPUBackend`.
- When results look wrong, compare with `ReferenceBackend()`: it is the definition the others are tested against.